Implementation Document

for

Academic & Schedule Management System (ASMS)

Version 0.1

Prepared by

Group #:23 **Group Name:** The Avengers

Samith Reddy	210797	reddysp21@iitk.ac.in
Abhishek Sardar	240041	abhishekh24@iitk.ac.in
Deepak	240327	deepakmg24@iitk.ac.in
Rajeswari K	240505	krajeswari24@iitk.ac.in
Sana Uday Wankhede	240924	sanauday24@iitk.ac.in
Tandel Darshan Navinkumar	241084	darshantn24@iitk.ac.in
Bharath Chandra Darla	240323	bharathc24@iitk.ac.in
Dharavath Shailaja	240348	shailajad24@iitk.ac.in
Siddharth S Godolkar	241015	siddharth24@iitk.ac.in

Course: CS253

Mentor TA: Aditya Ojha (adityaojha25@iitk.ac.in)

CONTENTS II
REVISIONS II

1 IMPLEMENTATION DETAILS 1

2 CODEBASE 2

3 COMPLETENESS 5

APPENDIX A - GROUP LOG 6

Revisions

Version	Primary Author(s)	Description of Version	Date Completed
0.1	Samith Reddy Abhishek Sardar Deepak Rajeshwari K Sana Uday Wankhede Tandel Darshan Navinkumar Bharath Chandra Darla Dharavath Shailaja Siddharth S Godolkar	First Draft	27/03/2026

1 Implementation Details

The ASMS project utilizes a **Polyglot Microservice Architecture**. This approach allows each service to use the optimal technology for its specific domain (e.g. Go for high-performance I/O, python for data analytics).

1.1 Presentation Tier

- **Technologies:** React 18+(Vite), Tailwind CSS, Lucide React, Framer Motion.
- **Justification:**
  - **React + Vite:** React's component-based architecture is essential for our "Atomic Design" system (Atoms,Molecules,Organisms). Vite was chosen over Create React App for significantly faster hot-module replacement and build times.
  - **Zustand:** Chosen for state management because it provides a lighter, less boilerplate-heavy alternative to Redux, which is ideal for managing our global ***authStore***.
  - **Tailwind CSS:** Enables a utility-first approach that ensures the dark-themed, tech focused aesthetic (Zinc/Gray/Blue) remains consistent across all 11 core pages.

1.2 Logic Tier

The backend is split into four specialized services:

- **Auth Service (Node.js/Express):** Handles identity management and JWT emission. Node.js is ideal for I/O-intensive authentication flows.
- **Academic Service (Python/Django):** Manages the core relational data of courses and enrollments. Django's robust ORM and built-in Admin panel accelerate the management of institutional data.
- **Productivity Service (Go/Gin):** Manages high-traffic submission nodes and discussion threads. Go's concurrency model (Goroutines) allows the system to handle thousands of simultaneous student submissions with minimal latency.
- **Analytics Service (Python/FastAPI):** Calculates GPA trends and identifies at risk students. FastAPI provides high performance while maintaining access to Python's data science libraries.

1.3 Data Tier

- **Primary System:** PostgreSQL 15.
- **Justification:** PostgreSQL was selected as the unified relational database for all services. Its support for UUIDs and robust ACID compliance ensures that a student ID generated by the Auth Service remains consistent and integral when referenced in the Academic or Productivity services.

1.4 Development and Version Control

- **Git & Github:** Used for source code management.
- **Docker & Docker Compose:** Containerization ensures that the diverse technical stack (Go,Node,Python) runs identically on all developers' machines. It manages the networking between the four microservices and the PostgreSQL cluster.

2 Codebase

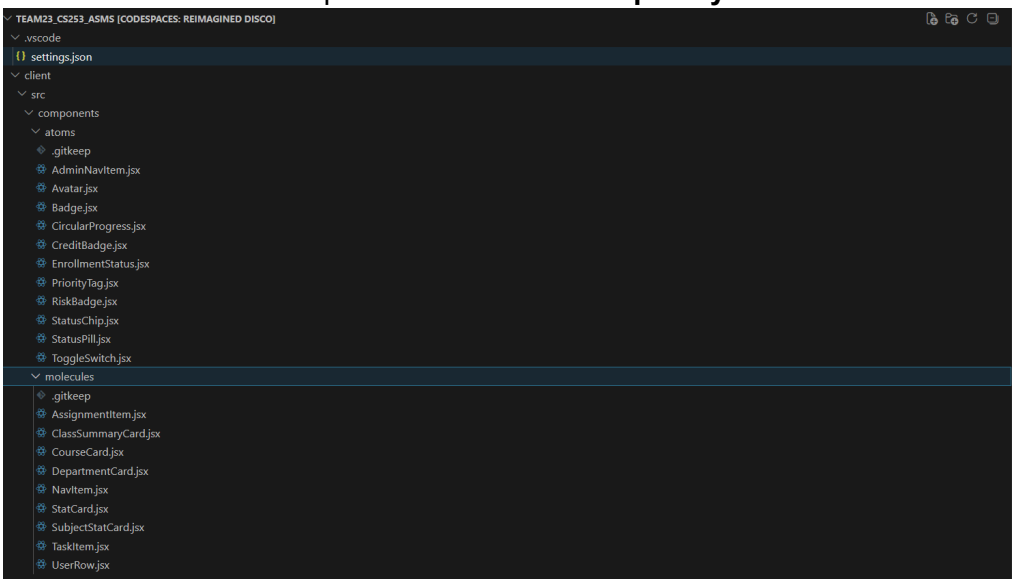
Repository Link

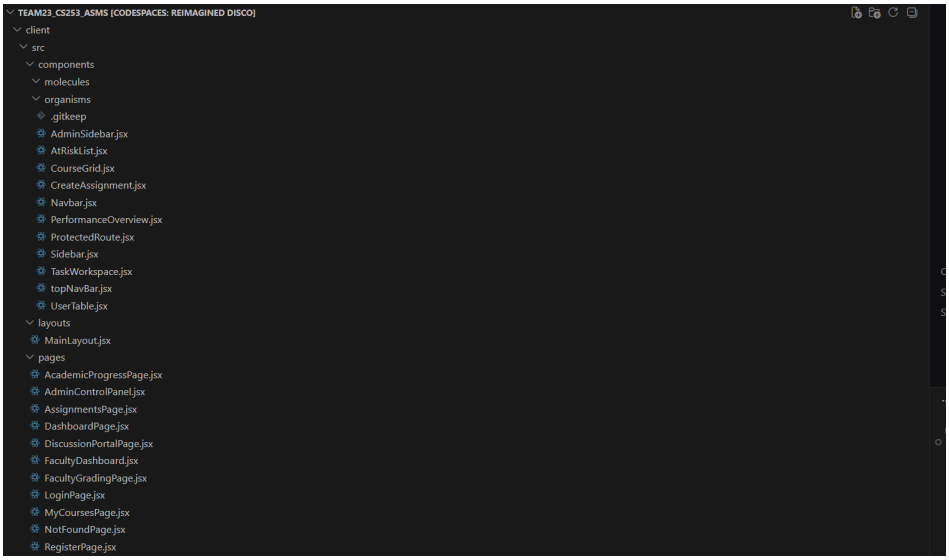
https://github.com/DarShan3115/TEAM23_CS253_ASMS.git

Codebase Navigation

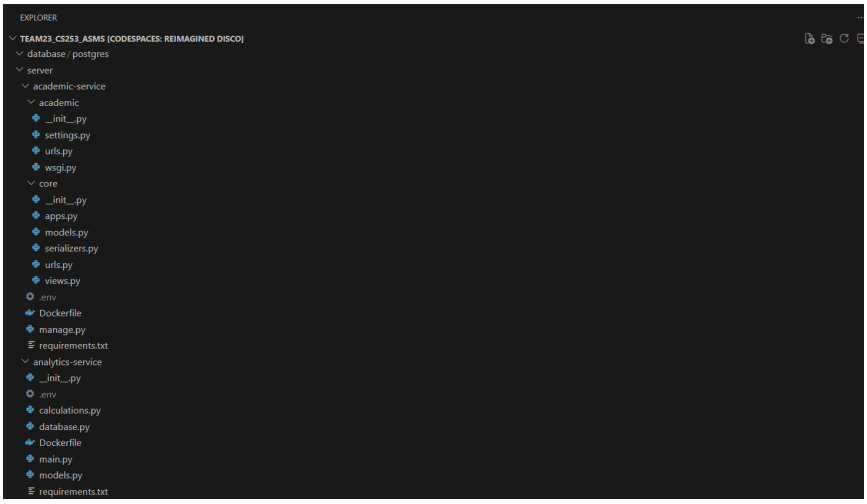
The codebase is structured to separate concerns between the frontend client and the backend server:

- **/client:** The React application root.
  - **/src/pages:** Contains 11 functional views.
  - **/src/components:** Divided into atoms, molecules and organisms following Atomic Design.
  - **src/store:** Contain the global Zustand state (authStore.js)
- **/server:** The microservice cluster.
  - **/auth-service:** Node.js identity provider.
  - **/academic-service:** Django academic record hub.
  - **/productivity-service:** Go-based submission and forum node.
  - **/analytics-service:** FastAPI performance analysis engine.
- **/database:** Contains **init.sql** for cross-service table initialization.
- **Root:** Contains **.env** templates and **docker-compose.yml** for orchestration.

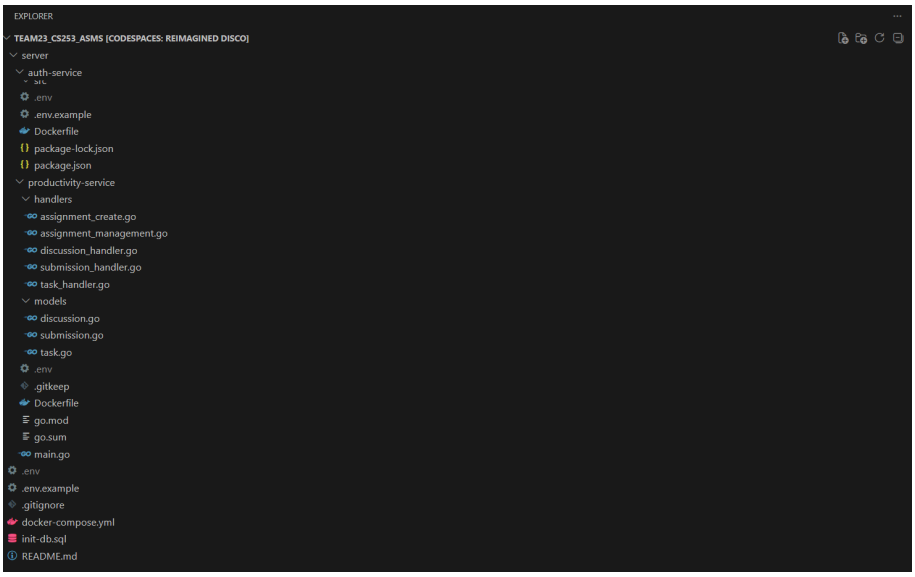



TEAM23_CS253_ASMS [CODESPACES: REIMAGINED DISCO]
client
src
components
molecules
organisms
.gitkeep
AdminSidebar.jsx
AtRiskList.jsx
CourseGrid.jsx
CreateAssignment.jsx
Navbar.jsx
PerformanceOverview.jsx
ProtectedRoute.jsx
Sidebar.jsx
TaskWorkspace.jsx
topNavBar.jsx
UserTable.jsx
layouts
MainLayout.jsx
pages
AcademicProgressPage.jsx
AdminControlPanel.jsx
AssignmentsPage.jsx
DashboardPage.jsx
DiscussionPortalPage.jsx
FacultyDashboard.jsx
FacultyGradingPage.jsx
LoginPage.jsx
MyCoursesPage.jsx
NotFoundPage.jsx
RegisterPage.jsx

EXPLORER
TEAM23_CS253_ASMS [CODESPACES: REIMAGINED DISCO]
client
src
pages
services
api.js
store
authStore.js
App.jsx
index.css
main.jsx
.env
Dockerfile
index.html
package-lock.json
package.json
postcss.config.js
tailwind.config.js
vite.config.js
database / postgres
init.sql


EXPLORER
TEAM23_CS253_ASMS [CODESPACES: REIMAGINED DISCO]
database / postgres
server
academic-service
academic
__init__.py
settings.py
urls.py
wsgi.py
core
__init__.py
apps.py
models.py
serializers.py
urls.py
views.py
.env
Dockerfile
manage.py
requirements.txt
analytics-service
__init__.py
.env
calculations.py
database.py
Dockerfile
main.py
models.py
requirements.txt

TEAM23_CS253_ASMS [CODESPACES: REIMAGINED DISCO]
server
analytics-service
auth-service
node_modules
src
config
db.js
controllers
admin.controller.js
auth.controller.js
users.controller.js
middleware
auth.js
routes
admin.routes.js
auth.routes.js
users.routes.js
app.js
.env
.env.example
Dockerfile
package-lock.json
package.json


EXPLORER
TEAM23_CS253_ASMS [CODESPACES: REIMAGINED DISCO]
server
auth-service
.env
.env.example
Dockerfile
package-lock.json
package.json
productivity-service
handlers
assignment_create.go
assignment_management.go
discussion_handler.go
submission_handler.go
task_handler.go
models
discussion.go
submission.go
task.go
.env
.gitkeep
Dockerfile
go.mod
go.sum
main.go
.env
.env.example
.gitignore
docker-compose.yml
init-db.sql
README.md

3 Completeness

SRS Parts Completed

1. **Identity Management:** Implemented the full **LoginPage** and **RegisterPage** with Role-Based Access Control (RBAC). The system correctly differentiates between Students, Faculty, and Admins.
2. **Course Management:** The **MyCoursesPage** allows for catalog browsing and enrollment, syncing with the Academic Service.
3. **Assignment Workflow:** The **AssignmentsPage** and **FacultyGradingPage** handle the full lifecycle of a task-from professor creation to student submission and final feedback.
4. **Academic Analytics:** The **AcademicProgressPage** visualizes GPA trends and attendance logs processed by the Analytics Service.
5. **Anonymous Collaboration:** The **DiscussionPortalPage** allows for the encrypted, anonymous student posts while verifying faculty responses.

Future Development Plan

- **AI Tutoring:** Implementing an automated feedback node using Gemini API to analyse student code/essays before they reach the Faculty Grading room.
- **Mobile Companion:** Developing a React Native application to provide push notifications for deadlines and grades.

Appendix A - Group Log

We've been communicating via **WhatsApp** group throughout the development of this project where we discuss each others' issues and finalize the code and decisions. Also this applies same to this document.